جامعة القدس
Al-Quds University

Al Quds University
Faculty of Computer Engineering

Special Topics - Section 1

Smart Home Emergency Alert System Using MQTT

Project Report

Student Name	Othman Othman
Student ID	22212228
Supervised by	Dr. Isam ishaq
Date	13 May 2026

Prepared for a practical IoT implementation using ESP32, Wokwi, PlatformIO, MQTT, Python, and Telegram Bot API.

Abstract

This report presents the design and implementation of a smart home emergency alert system based on the MQTT communication protocol. The system uses an ESP32 microcontroller connected to a PIR motion sensor and a DHT22 temperature sensor. When motion is detected or the temperature reaches 40 degrees Celsius or higher, the system activates a local alarm using an LED and buzzer. In parallel, the ESP32 publishes emergency messages to an MQTT broker. A Python MQTT subscriber bridge receives the emergency messages and forwards them to a Telegram bot as real-time notifications. The project demonstrates a complete IoT data flow from sensing and embedded decision-making to MQTT communication and remote alerting.

Keywords

ESP32, MQTT, IoT, PIR sensor, DHT22, Telegram Bot, Python, Wokwi, PlatformIO, real-time alerting.

1. Introduction

Smart home systems require fast and reliable emergency monitoring. Two common safety conditions in residential environments are unauthorized motion and abnormal temperature increase. This project implements a compact IoT-based emergency alert system that can detect both conditions, activate a local alarm, and send remote notifications using MQTT-based communication.

The main purpose of the project is not only to connect sensors to a microcontroller, but also to demonstrate a complete IoT architecture. The ESP32 acts as the embedded sensing and decision unit, MQTT acts as the main communication protocol, and Telegram acts as the notification interface for the user.

2. Project Objectives

- Design an ESP32-based emergency monitoring system for a smart home environment.
- Detect motion using a PIR motion sensor.
- Monitor temperature using a DHT22 sensor and trigger an alert at 40 degrees Celsius or higher.
- Activate a local alarm using an LED and buzzer.
- Use MQTT publish/subscribe communication to send emergency alerts.
- Forward MQTT emergency messages to Telegram through a Python subscriber bridge.
- Verify the system behavior using Wokwi simulation and VS Code/PlatformIO.

3. System Components

Component	Role in the System
ESP32	Main controller that reads sensors, applies decision logic, controls the alarm, and publishes MQTT messages.
PIR Motion Sensor	Detects motion and sends a digital HIGH signal to the ESP32 when motion is detected.
DHT22 Sensor	Measures temperature and provides environmental data to the ESP32.
LED	Visual local alarm indicator.
Buzzer	Audio local alarm indicator.
MQTT Broker	Receives published messages from ESP32 and distributes them to subscribers.
Python Bridge	Subscribes to the emergency topic and forwards alerts to Telegram.
Telegram Bot	Displays real-time emergency notifications to the user.

4. System Architecture

The system is divided into three main layers: the embedded sensing layer, the MQTT communication layer, and the notification layer. The embedded layer includes the ESP32 and sensors. The communication layer is based on MQTT, where the ESP32 publishes messages to a broker. The notification layer is implemented using a Python subscriber that forwards emergency alerts to Telegram.

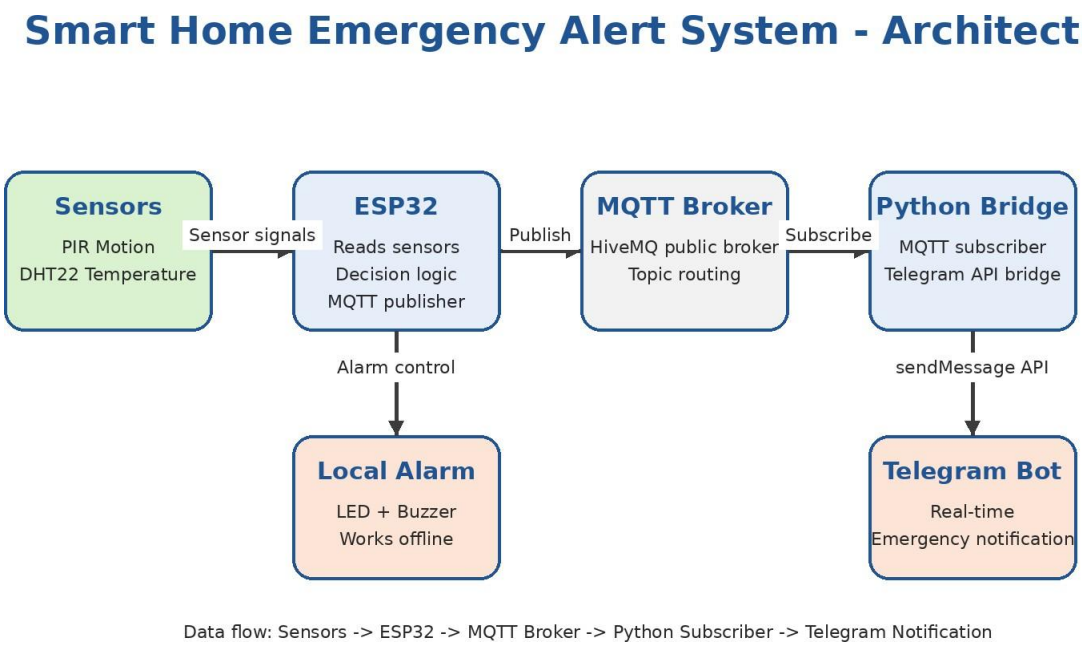

Figure 1. Overall system architecture and data flow.

4.1 MQTT Publish/Subscribe Model

MQTT is the main IoT communication protocol in this project. The ESP32 works as an MQTT publisher, while the Python bridge works as an MQTT subscriber. The broker used in the implementation is broker.hivemq.com. The ESP32 does not communicate directly with Telegram. Instead, it publishes alert messages to MQTT topics, and the Python subscriber receives those messages and forwards them to Telegram.

MQTT Topic	Purpose
othman/home/security/motion	Motion detection messages.
othman/home/environment/temperature	Temperature readings from DHT22.
othman/home/emergency/alert	Critical emergency alerts received by the Python bridge.
othman/home/system/status	System startup and status messages.

5. Hardware and Circuit Design

The circuit was implemented and tested using Wokwi simulation. The ESP32 is connected to the PIR sensor, DHT22 sensor, LED, and buzzer. The local alarm is controlled directly by the ESP32, which means that the LED and buzzer can still operate even if MQTT or internet connectivity fails.

Device	ESP32 Pin	Function
PIR Motion Sensor OUT	GPIO 27	Digital motion input.
DHT22 DATA	GPIO 15	Temperature sensor data input.
Buzzer	GPIO 26	Audio alarm output using ESP32 PWM/LEDC.
Red LED	GPIO 25	Visual alarm output through a resistor.

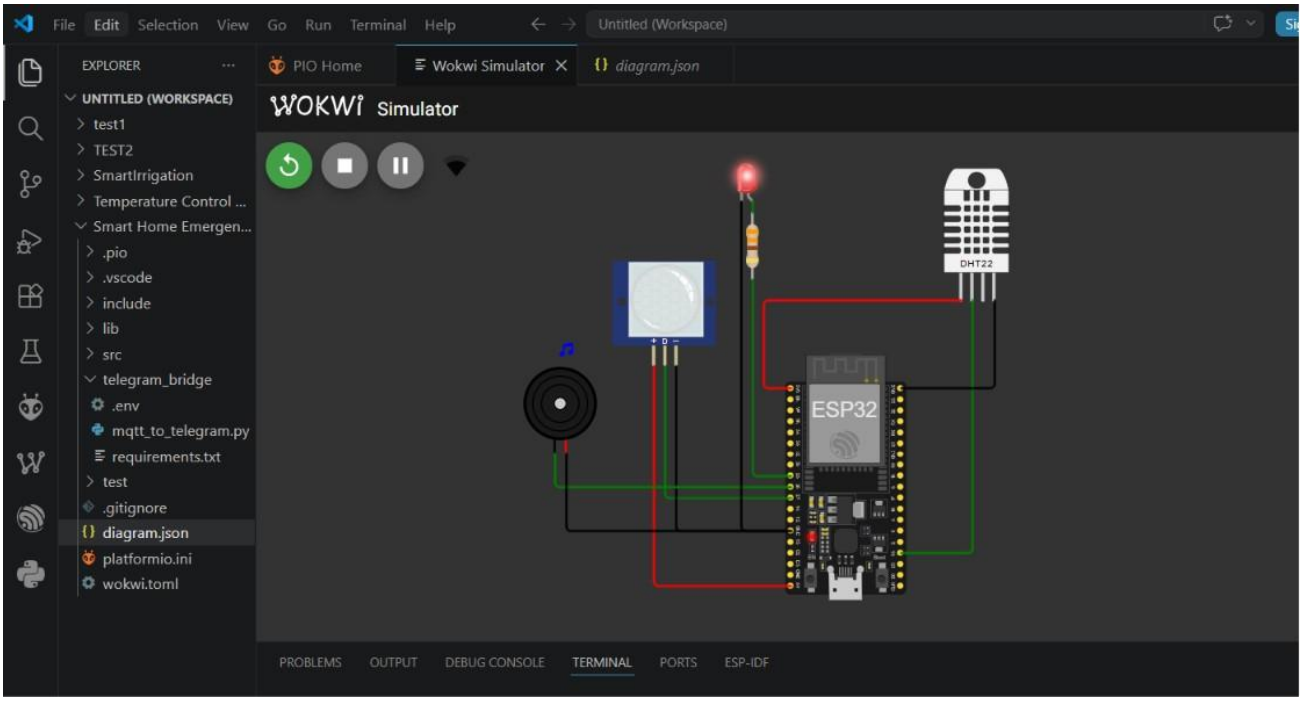

Figure 2. Wokwi circuit simulation showing ESP32, PIR sensor, DHT22, LED, and buzzer.

6. Software Implementation

The software implementation consists of two main parts: ESP32 firmware and a Python MQTT-to-Telegram bridge.

6.1 ESP32 Firmware

The ESP32 firmware was developed using the Arduino framework in PlatformIO. It connects to Wokwi WiFi, establishes an MQTT connection, reads sensor values every two seconds, controls the alarm, and publishes messages to MQTT topics. The temperature threshold is set to 40 degrees Celsius. If the temperature is 40 degrees or higher, the local alarm remains active. When the temperature falls below 40 degrees, the temperature alarm turns off. For motion detection, the local alarm is activated for five seconds for each motion event.

Condition	System Response
Temperature >= 40 degrees Celsius	LED and buzzer stay ON; overtemperature alert is published to MQTT.
Temperature < 40 degrees Celsius	Temperature alarm is turned OFF unless motion alarm is active.
Motion detected	LED and buzzer turn ON for five seconds; motion alert is published to MQTT.
MQTT offline	Local alarm still works; remote notification is delayed or unavailable.

6.2 Python MQTT-to-Telegram Bridge

The Python bridge uses the paho-mqtt library to subscribe to the emergency alert topic. When a message is received, the script sends the message to Telegram using the Telegram Bot API. Configuration values

such as BOT_TOKEN, CHAT_ID, MQTT_BROKER, MQTT_PORT, and MQTT_TOPIC are stored in a .env file. The bot token is treated as a secret and must not be shared publicly.

```
BOT_TOKEN=<secret_token>
CHAT_ID=6111045839
MQTT_BROKER=broker.hivemq.com
MQTT_PORT=1883
MQTT_TOPIC=othman/home/emergency/alert
```

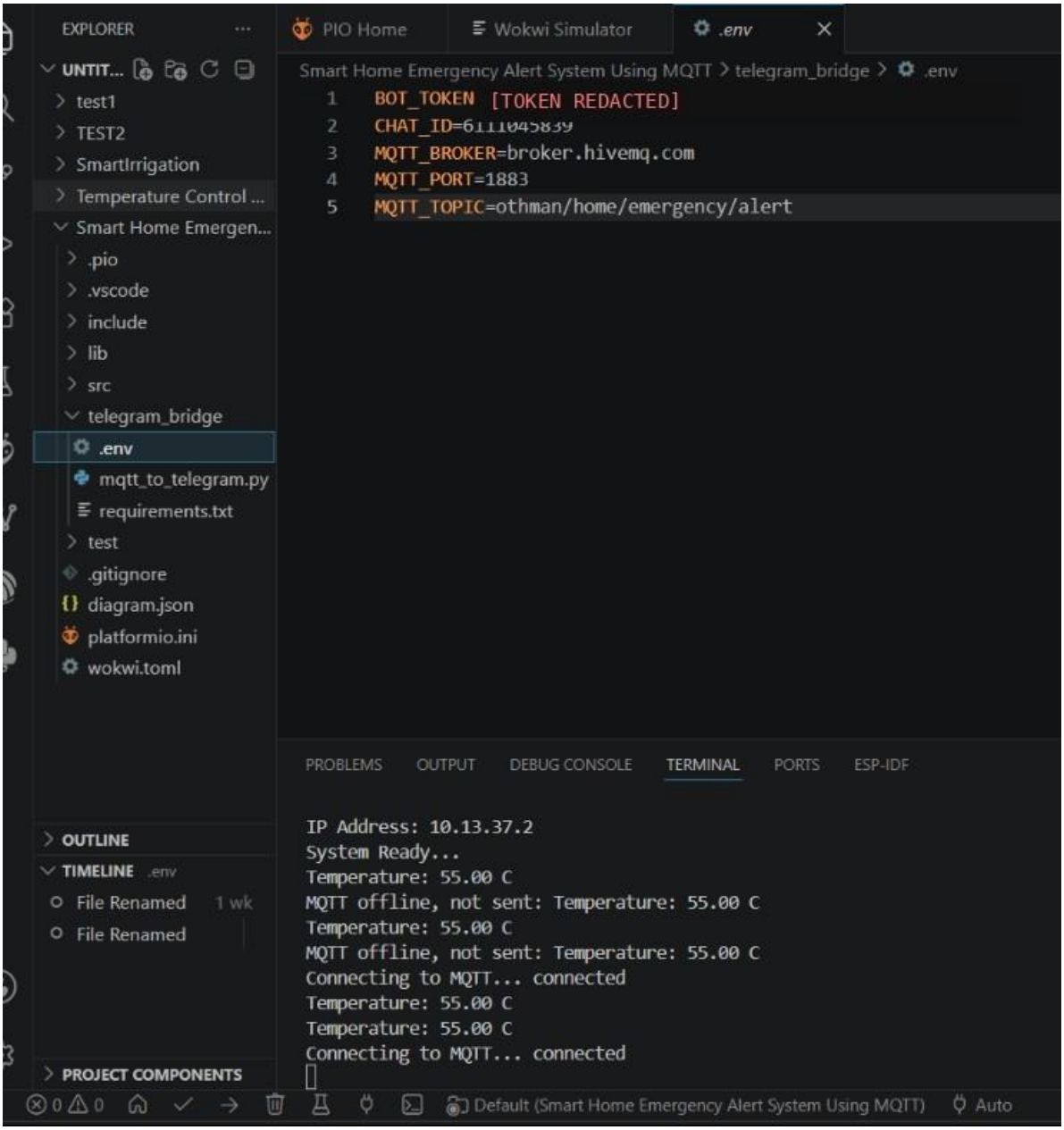

Figure 3. Python bridge configuration and terminal output. The bot token is redacted for security.

7. Testing and Results

The system was tested through Wokwi simulation and Telegram notifications. The test confirmed that the ESP32 reads the sensor values, detects emergency conditions, activates the local alarm, publishes MQTT messages, and sends Telegram notifications through the Python bridge.

7.1 Overtemperature Test

During the overtemperature test, the DHT22 temperature value was increased above the 40 degrees Celsius threshold. The ESP32 detected the abnormal temperature, activated the LED and buzzer, and published an MQTT emergency message. The Python bridge received the message and forwarded it to Telegram.

7.2 Motion Detection Test

During the motion detection test, the PIR sensor was triggered. The ESP32 detected the motion event, activated the alarm for five seconds, and sent an intruder motion alert through MQTT. The Telegram bot received the notification successfully.

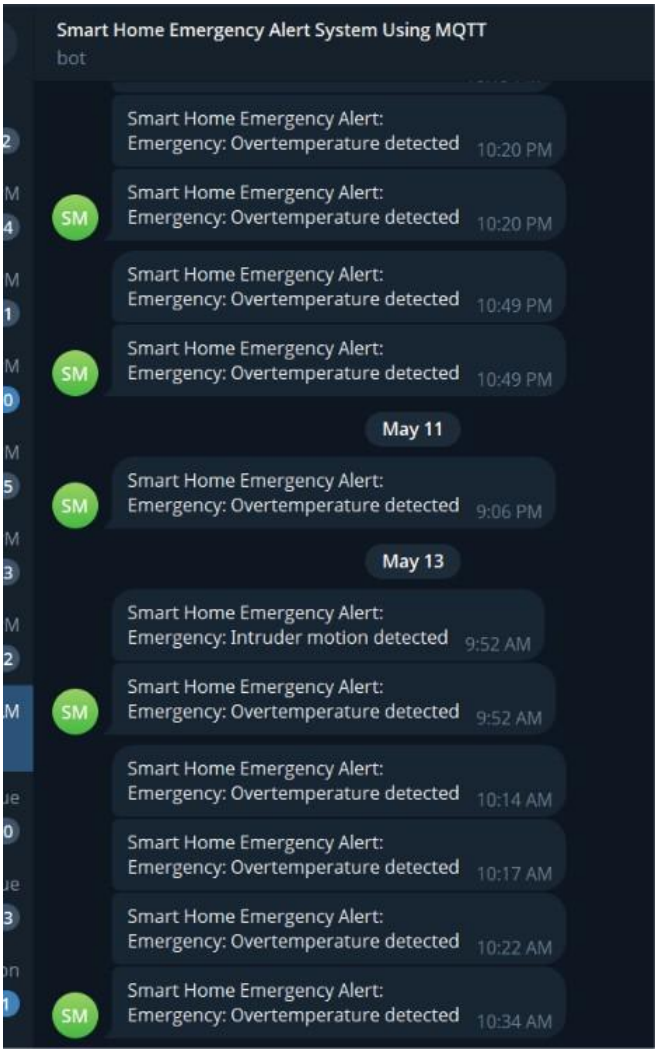

Figure 4. Telegram bot receiving real-time emergency notifications.

8. Discussion

The project demonstrates a practical IoT architecture with clear separation between sensing, communication, and notification. The local alarm is handled directly by the ESP32, while the remote notification is handled through MQTT and Python. This separation improves reliability because the safety alarm does not depend completely on the MQTT connection. If the MQTT broker is unavailable, the LED and buzzer can still respond to emergency conditions locally.

The most important engineering concept in the project is the MQTT publish/subscribe model. It allows the ESP32 to publish messages without needing to know the final receiver. Any subscriber listening to the correct topic can receive the emergency message. In this implementation, the subscriber is a Python bridge that forwards alerts to Telegram.

9. Limitations

- The project was tested in Wokwi simulation rather than on a physical hardware prototype.
- A public MQTT broker was used, so the implementation does not include authentication or TLS security.
- The system currently focuses on emergency alerting and local alarm control, not automatic environmental control such as turning on a fan or ventilation system.
- The Telegram notification depends on the Python bridge being active on the computer.

10. Conclusion

This project successfully implements a smart home emergency alert system using ESP32 and MQTT. The system detects motion and high temperature, activates local alarms, publishes emergency alerts to MQTT topics, and forwards critical alerts to Telegram through a Python bridge. The project combines embedded systems, sensor interfacing, MQTT communication, Python automation, and real-time alerting in a clear and practical IoT architecture.

References

[1] MQTT protocol documentation and publish/subscribe communication model.

[2] ESP32 Arduino framework and PlatformIO development environment documentation.

[3] Wokwi ESP32 simulation environment documentation.

[4] Python paho-mqtt library documentation.

[5] Telegram Bot API documentation.